

School of Informatics



Natural Language Understanding, Generation, and Machine Translation

Coursework 1: Recurrent Neural Networks

Date: Wednesday 26th February, 2025

1 Language Modeling

In this section we present the results of the training and tuning a Recurrent Neural Network (RNN) for language modeling, using backpropagation through time (BPTT).

1.1 Parameter Tuning

We begun with the tuning of model parameters, specifically the learning rate, the number of hidden units and the number of steps to lock back in the truncated backpropagation. We explored all possible configurations of the following parameter values:

- Learning rate = 0.05, 0.1, 0.5
- Hidden units = 25, 50.
- BPTT steps = 0, 2, 5.

The experiment was carried out using a training set of 1,000 sentences and a development set of 1,000 sentences, taken from the *wiki-train.txt* and *wiki-dev.txt* provided files respectively. The training was carried out for 10 epochs and a fixed vocabulary of 2,000 tokens was used. The tuning was applied with the aim of minimising the cross-entropy loss on the development set.

Table 1 below presents the outcomes from each RNN training session conducted during the parameter tuning phase. It details the best cross-entropy loss achieved after 10 epochs and the duration of the training in seconds.

Hidden Units	Learning Rate	BPTT Steps	Best Cross-Entropy Loss	Training Time (s)
25	0.5	0	5.017	241.61
25	0.5	2	4.978	265.74
25	0.5	5	4.933*	297.32
25	0.1	0	5.196	241.16
25	0.1	2	5.191	265.30
25	0.1	5	5.210	296.56
25	0.05	0	5.359	233.03
25	0.05	2	5.381	257.71
25	0.05	5	5.373	292.12
50	0.5	0	4.970	263.39
50	0.5	2	5.028	306.62
50	0.5	5	4.965	364.31
50	0.1	0	5.152	262.22
50	0.1	2	5.124	305.48
50	0.1	5	5.141	361.13
50	0.05	0	5.267	254.69
50	0.05	2	5.285	297.22
50	0.05	5	5.316	353.73

Table 1: Parameter Tuning for hidden units, learning rates, and BPTT steps (* lowest loss found).

As shown above, setting the number of hidden units to 25, the learning rate to 0.5 and BPTT steps to 5, resulted in a cross-entropy loss of 4.933 which was the lowest achieved throughout, with the training taking a total of 297.32 seconds. Additionally, a noticeably low cross-entropy loss of 4.965 was achieved for 50 hidden units, 0.5 leaning rate and 5 BBPT steps.

As shown in Table 1, higher learning rates of 0.5 consistently yielded lower cross-entropy losses across all configurations of hidden units and BPTT steps. This suggests that the larger learning rates facilitate faster convergence by enabling more substantial updates to the weights during backpropagation. While such large updates can lead to instability or overshooting the optimal weights, the results indicate that this was not a significant issue within the tested parameter range, suggesting a well-balanced learning rate for the given model architecture and task. On the other hand, the smaller learning rates of 0.1 and 0.05 appeared to be less effective, possibly failing to escape local minima or to adequately explore the solution space within the limited epochs provided. Shifting focus to the model’s architecture, increasing the number of hidden units did not reliably enhance the reduction of cross-entropy loss, as evidenced in Table 1. While additional hidden units theoretically increase the model’s capacity to learn more complex patterns, the data suggests that the complexity introduced may not have been necessary for the task at hand. This is indicated by the lack of significant performance enhancement with 50 hidden units compared to 25. Such an increase may lead to inefficiencies and potential overfitting, especially without a corresponding increase in the complexity or size of the training data. Lastly, enhancing the model with increased BPTT steps generally resulted in improved performance, particularly notable at higher learning rates. Extending BPTT steps enhances the model’s capability to capture longer dependencies within the sequence data, crucial for effective language modeling. However, this benefit also comes with increased computational demands and extended training times. The varying results across different configurations suggest complex interactions between the depth of BPTT, the model’s architecture, and the data’s characteristics, pointing to a delicate balance between capturing long-range dependencies and managing computational resources efficiently.

1.2 Model Training

Following the parameter tuning, the best performing parameters for learning rate (0.1), hidden units (25) and BPTT steps (5) were used to train the RNN model. This time the model was trained with a dataset of 25,000 sentence from the *wiki-train.txt* file, while 1,000 sentences were still used for the development set, to evaluate the models performance at each epoch. Once again the model was trained for 10 epochs with a vocabulary size of 2,000 tokens being used.

The training of the model resulted on a final development cross-entropy loss of 4.459 as shown in table 2. This is an improvement on the loss achieved with the same configurations during the previous section by 0.474. This decrease in loss is expected as we are now training the model on a far larger training set. This exposes the model to a broader array of linguistic patterns and contexts, enhancing its ability to generalize better to unseen data.

Following the training, the model was evaluated on the *wiki-test.txt* test set provided for the assignment. On this test set, the model achieved a mean loss of 4.463, corresponding to a perplexity of 86.78 as shown in table 2. The model's performance on the test set is only marginally worse than on the development set, with a difference of 0.003 in mean loss, demonstrating effective generalization and a low risk of overfitting. Meanwhile, the perplexity of 86.78 suggests that while the model predicts the next word in a sequence with reasonable accuracy, there remains potential for further improvements.

Development Set Evaluation	Test Set Evaluation	
Final Run Loss	Mean Loss	Perplexity
4.459	4.463	86.78

Table 2: Results of model implementation with optimized parameters on the test dataset.

2 Predicting Agreement with RNNs and GRUs

This section aims to analyze the effectiveness of RNN and GRU architectures on a number prediction task, with the backpropagation through time (BPTT) steps set to 0, which indicates that the model is truncated to look only one step behind. The key focus is to evaluate the change in loss, accuracy, and training time under different hidden unit sizes (10, 25, and 50).

The validation loss and accuracy of RNN and GRU are summarized in Table 3 and Table 4, reporting results at key epochs and for the final trained model (determined by lowest validation loss). Table 5 presents the average and total training time. The full dataset, including validation loss and accuracy for all epochs, is available in Appendix A.

2.1 Loss

	Hidden	Initial	Epoch 5	Epoch 10	Best Epoch	Best
RNN	10	0.6486	0.6291	0.6161	10	0.6161
	25	0.6586	0.6250	0.6097	10	0.6097
	50	0.7466	0.6341	0.5975	10	0.5975
GRU	10	0.6957	0.6265	0.5918	10	0.5918
	25	0.8096	0.5692	0.5467	10	0.5467
	50	0.7328	0.5433	0.5234	10	0.5234*

Table 3: Validation Loss for RNN and GRU with Varying Hidden Units at Key Epochs

The results show that, across both RNN and GRU models, increasing the number of hidden units reduces the final loss. A greater number of hidden units enhances the model’s capacity to learn complex representations, thereby improving generalization. Larger hidden layers allow the network to extract more abstract features, ultimately leading to better performance. Conversely, with only 10 hidden units, the model lacks sufficient capacity to effectively distinguish key features, limiting its ability to capture meaningful patterns. Additionally, since backsteps = 0, the model is restricted to looking only one step back, making hidden unit size a more crucial determinant of its learning capacity.

The best loss is achieved at the last epoch, which may suggest that the model is underfitted due to an insufficient number of training epochs or a slow learning rate.

Moreover, GRU models consistently outperform RNNs in terms of final loss values across all hidden unit configurations. GRU models exhibit smoother convergence patterns, indicating improved stability in capturing long-range dependencies. Conversely, the RNN with 50 hidden units demonstrates the highest loss fluctuation, followed by the RNN with 25 hidden units and other models. This suggests that GRUs provide superior training stability, likely due to their update gate mechanism, which regulates the retention of past information and controls the extent of model updates during training.

2.2 Accuracy

	Hidden	Initial	Epoch 5	Epoch 10	Final
RNN	10	0.659	0.659	0.659	0.659
	25	0.656	0.664	0.688	0.688
	50	0.409	0.669	0.687	0.687
GRU	10	0.511	0.666	0.672	0.672
	25	0.380	0.718	0.734	0.734
	50	0.531	0.738	0.751	0.751*

Table 4: Accuracy for RNN and GRU with Varying Hidden Units at Key Epochs

The results indicate that the number of hidden units improves accuracy for both GRU and RNN models. Larger hidden layers enable the models to capture more complex dependencies, leading to enhanced classification performance.

For RNNs, the best-performing model, RNN-50, achieves 68.7% accuracy, which is significantly higher than RNN-10 at 65.9%. Notably, RNNs with only 10 hidden units show no improvement, highlighting their limited capacity for learning temporal dependencies. A similar trend is observed in GRU models, where GRU-50 reaches 75.1% accuracy, outperforming GRU-10 at 67.2%.

The performance improvement due to increased hidden units is more pronounced in GRUs than in RNNs, indicating that GRUs make more effective use of additional capacity. The gating mechanisms in GRUs enable them to filter out irrelevant information and retain useful context, even when memory is limited. As hidden layer size increases, GRUs efficiently utilize the extra capacity, whereas RNNs struggle due to the absence of memory control mechanisms.

Across all hidden unit configurations, GRUs achieve higher accuracy than RNNs. This can be attributed to GRUs’ ability to manage state transitions more efficiently using gating mechanisms, which allows them to model sequential data more effectively—even when restricted to looking only one step back.

2.3 Time Taken

	Hidden	Avg. Per Epoch (s)	Total Time Taken (s)
RNN	10	1.46	15.20
	25	2.40	24.92
	50	4.41	45.33
GRU	10	3.94	40.85
	25	7.01	72.93
	50	8.27	85.78

Table 5: Training Time Summary for RNN and GRU with Varying Hidden Units

GRUs require significantly more training time than RNNs due to the additional computations involved in their gated mechanisms.

The training time increases quadratically with the number of hidden units, as each additional unit contributes to an increase in matrix multiplications, thereby escalating the computational cost (Appendix B).

However, these results are subject to hardware constraints, particularly the availability and performance fluctuations of the CPU at the time of training.

3 Comparing Recurrent Models

This section aims to analyze the effect of Backpropagation Through Time (BPTT) step size on the performance of RNNs and GRUs by evaluating its impact on perplexity, accuracy, and gradient flow. Additionally, it investigates the role of GRU’s reset and update gates by selectively disabling them to assess their individual contributions to model performance.

Experiment 1: Investigate how varying BPTT step sizes (5, 10, 20, 30, 50) affects perplexity, accuracy, and gradient flow in RNNs and GRUs.

Experiment 2: Assess the contribution of reset and update gates in GRUs by selectively disabling them.

3.1 Hypothesis

Experiment 1

Increasing the number of Backpropagation Through Time (BPTT) steps will initially improve model performance for both RNN and GRUs, but beyond a certain threshold, the improvement will plateau. The effectiveness of BPTT comes from its ability to propagate gradients across multiple time steps, allowing models to learn dependencies between distant elements in a sequence. However, increasing BPTT steps beyond a certain point does not yield substantial improvements for three key reasons:

Sentence length constraint – Most sentences in the dataset have a natural length boundary (around 30 tokens), beyond which additional BPTT steps do not contribute meaningful new information.

Computational cost – Increasing BPTT steps significantly raises computational complexity, making training more expensive without proportionally improving accuracy or perplexity.

Localized dependencies in number agreement – The verb in a sentence typically depends only on the subject, which is usually found near the beginning of the sentence. Also, extending BPTT beyond the relevant range can expose the model to unnecessary or misleading information, such as intervening clauses, modifiers, or unrelated sentence components. Since number agreement might not require information from the entire sentence, capturing dependencies beyond the relevant context provides little benefit.

Thus, we expect that a moderate number of BPTT steps (e.g., 10–15) will be optimal, while further increases beyond this range (e.g., 30–50) will lead to diminishing returns.

BPTT Steps	Expected Effect
0	No backpropagation through time.
5	Captures short-term dependencies but may miss long-range relations.
10, 15	Best balance between stability and performance.
20, 30	Shows diminishing returns.
50	Investigated as an extreme case.

Table 6: Effect of different BPTT step sizes

Experiment 2

GRUs outperform RNNs primarily due to their gating mechanisms, which enable better control over information flow and help mitigate the vanishing gradient problem. If these gates are removed, the GRU will lose its advantage and perform similarly to or worse than a standard RNN, as it will no longer be able to regulate memory retention and state updates effectively.

Specifically, we hypothesize that removing the update gate will have a more severe impact on performance than removing the reset gate, as the update gate is the gate that controls long-term memory retention and is the key to preventing vanishing gradients.

Expected Impact on Training

Gate Disabled	Expected Effect
Reset Gate Off	Retains full past states, improving long-range dependencies but reducing flexibility.
Update Gate Off	Model aggressively overwrites hidden states, leading to instability and overfitting.
Both Gates Off	GRU effectively collapses into a simple RNN, suffering from vanishing gradients.

Table 7: Expected Effects of Disabling Gates in GRUs

Removing the reset gate allows the GRU to retain more past information at each step. This could be beneficial for tasks requiring long-range dependencies but may also introduce redundant or irrelevant past states, reducing model flexibility.

In contrast, removing the update gate forces the GRU to overwrite its hidden state at every step, which can cause instability and excessive sensitivity to recent inputs. This is expected to increase overfitting and degrade performance on longer sequences.

When both gates are disabled, the model no longer has gating mechanisms, effectively reducing it to a basic RNN architecture. Since RNNs suffer from vanishing gradients in long-term dependencies, this setup is expected to result in the poorest performance among all configurations.

3.2 Experimental Setup

3.2.1 Experiment 1

Introduction

The main objective of this experiment is to investigate how varying Backpropagation Through Time (BPTT) step sizes influence:

- Perplexity
- Accuracy
- Gradient flow and vanishing gradient behavior

Modifications to the Training Pipeline

To support deeper analysis, the standard `runner.py` file was modified to create `RunnerQ4`, which introduces:

- Expanded logging for Perplexity and Accuracy at every epoch.
- Gradient norms through time logging, specifically:
 - $\|\Delta U\|$ (RNN)
 - $\|\Delta U_r\|, \|\Delta U_z\|, \|\Delta U_h\|$ (GRU)

To track gradient propagation, BPTT logging was implemented to monitor gradient norm changes over time. Specifically, the gradient norms of $\|\Delta U\|$ (RNN) or $\|\Delta U_r\|, \|\Delta U_z\|, \|\Delta U_h\|$ (GRU) were recorded for the first five sentences that underwent full backpropagation (long enough sequences) at the first epoch. These gradients were logged at every time step to analyze how gradients propagate and decay. To enable this, modifications were also made to `rnn.py` (`acc_deltas_bptt_np()`), `gru.py` (`acc_deltas_bptt_np()`), and `gru_abstract.py` (`backward()`) to facilitate gradient tracking.

Hyperparameter Settings

BPTT Step Size Selection The experiment tested $\text{BPTT} = \{0, 5, 10, 20, 30, 50\}$:

Number of Epochs and Training Samples As stated in Question 4, the number of epochs and training sentences were increased to enhance model performance.

The number of epochs was increased to ensure convergence, as RNNs, particularly those using BPTT, require more training iterations to stabilize due to the vanishing gradient problem. Extending the training duration allows the models to learn more effectively and achieve proper optimization. Early stopping, implemented in `runner.py`, prevented overfitting due to the large number of epochs.

Additionally, the training dataset size was expanded to 25,000 sentences to ensure that the models learn long-range dependencies rather than memorizing patterns from a small dataset. This increase helps improve generalization and prevents overfitting to limited training samples.

Initial Hyperparameter Tuning To select optimal parameters, we conducted a simple grid search with `lookback = 5`.

Findings:

- Higher learning rates (0.5) generally led to better performance, especially in GRUs.
- Lower learning rates (0.05) consistently resulted in lower accuracy, suggesting slower convergence.
- Increasing hidden units from 50 to 75 did not significantly improve accuracy in either model.

Model	Hidden Units	Learning Rate	Accuracy
RNN	50	0.5	0.696
RNN	50	0.1	0.665
RNN	50	0.05	0.662
RNN	75	0.5	0.690
RNN	75	0.1	0.665
RNN	75	0.05	0.665
GRU	50	0.5	0.768
GRU	50	0.1	0.708
GRU	50	0.05	0.665
GRU	75	0.5	0.752
GRU	75	0.1	0.682
GRU	75	0.05	0.665

Table 8: Hyperparameter search results

Given the computational cost associated with higher hidden dimensions, 50 hidden units provided a better balance between efficiency and performance.

Final Parameter Selection:

- **Hidden Units:** 50
- **Learning Rate:** 0.5
- **Epochs:** 40

Evaluation Metrics

To measure performance, the following metrics were tracked per epoch:

Perplexity

$$\text{Perplexity} = \exp(\text{cross-entropy loss}) \quad (1)$$

Accuracy

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}} \quad (2)$$

Gradient Norms To track gradient propagation and decay:

$$\|\Delta U\|, \|\Delta U_r\|, \|\Delta U_z\|, \|\Delta U_h\| \quad (3)$$

3.2.2 Experiment 2

Investigation of Reset and Update Gates in GRUs

This experiment investigates the role of reset and update gates in GRUs by selectively disabling them. The primary objective is to analyze how these modifications influence gradient propagation, accuracy, and perplexity during training. By removing one or both gates, we aim to understand their individual contributions to memory retention, information flow, and overall model effectiveness.

Unlike standard RNNs, GRUs utilize gating mechanisms to regulate how information is stored and updated in the hidden state. The reset gate controls how much past information should be discarded, while the update gate determines how much new information should overwrite the previous hidden state. By disabling these gates, we examine how the model behaves under constrained information flow conditions.

GRU Configurations To systematically assess the impact of gating mechanisms, we evaluated the following four GRU configurations:

Model Variant	Reset Gate	Update Gate
Baseline GRU	On	On
GRU (Reset Disabled)	Off	On
GRU (Update Disabled)	On	Off
GRU (Both Disabled)	Off	Off

Table 9: GRU Configurations with Disabled Gates

To support the gating analysis, mode selection was integrated into `RunnerQ4`, enabling toggling of the reset and update gates during training. Additionally, the `forward()` method in `gru.py` was modified to ensure correct handling of disabled gates.

Equations for Disabled Gates

When the reset gate is disabled, it is forced to be 1, meaning past information is fully retained at every step:

$$h_t = \tanh(W_h x_t + U_h (s_{t-1} \times 1)) \quad (4)$$

This modification prevents selective forgetting, making the GRU behave more like an LSTM, which retains historical context for longer sequences.

When the update gate is disabled, it is set to 0, meaning the hidden state is completely overwritten at each step:

$$s_t = (1 - 0)h_t + 0s_{t-1} = h_t \quad (5)$$

This removes selective state retention, causing the model to behave like a traditional RNN, which is prone to unstable updates and vanishing gradients over long sequences.

Hyperparameter Selection

To systematically assess the impact of gating mechanisms, we evaluated the following four GRU configurations:

Model Variant	Reset Gate	Update Gate
Baseline GRU	On	On
GRU (Reset Disabled)	Off	On
GRU (Update Disabled)	On	Off
GRU (Both Disabled)	Off	Off

Table 10: GRU Configurations for Gating Experiments

For a controlled comparison, we used the best-performing settings from Experiment 1:

- **BPTT Steps:** 10 (Selected based on Experiment 1, where $\text{BPTT} = 10$ provided the best performance)
- **Hidden Units:** 50
- **Learning Rate:** 0.5
- **Epochs:** 40
- **Training Sentences:** 25,000
- **Validation Sentences:** 1,000

These parameters ensure that the effect of gating modifications is isolated, without interference from external hyperparameter changes.

3.3 Analysis

3.3.1 Experiment 1

The results achieved through our first experiment, detailed in part in Figure 1, highlight a significant performance degradation for models configured with zero BPTT steps in both the RNN and GRU models, underscoring the critical role of temporal context in sequence learning models. Once again, for both the GRU and RNN models the configuration with 15 BPTT steps exhibited the best performance, achieving this at 25 epochs, the margin of superiority was notably slim. Therefore, while 15 BPTT steps may optimally balance efficiency with effective dependency capture, the performance across other configurations (from 5 to 50 BPTT steps) remained remarkably consistent. This indicates that minimal BPTT steps beyond zero are generally sufficient for capturing the necessary temporal dependencies, with incremental increases in BPTT steps offering only marginal benefits. Further analysis reveals that extending BPTT beyond the average sentence length of 30 tokens offers no performance gains. Pushing BPTT to 50 steps merely increases computational load without improving accuracy or perplexity, as sentences exceeding 30 tokens are rare and once the typical sentence boundary is reached, further extension only contributes to computational redundancy without adding value to model training. Therefore, any steps beyond this threshold fail to capture additional meaningful linguistic patterns or dependencies. Additionally, number agreement in verbs depends mainly on the words immediately before the verb, so extending BPTT beyond the relevant range may introduce unnecessary or misleading information. While the difference in performance of models with non-zero BPTT steps is minimal, it still supports our hypothesis that optimal performance would be achieved for 10-15 steps.

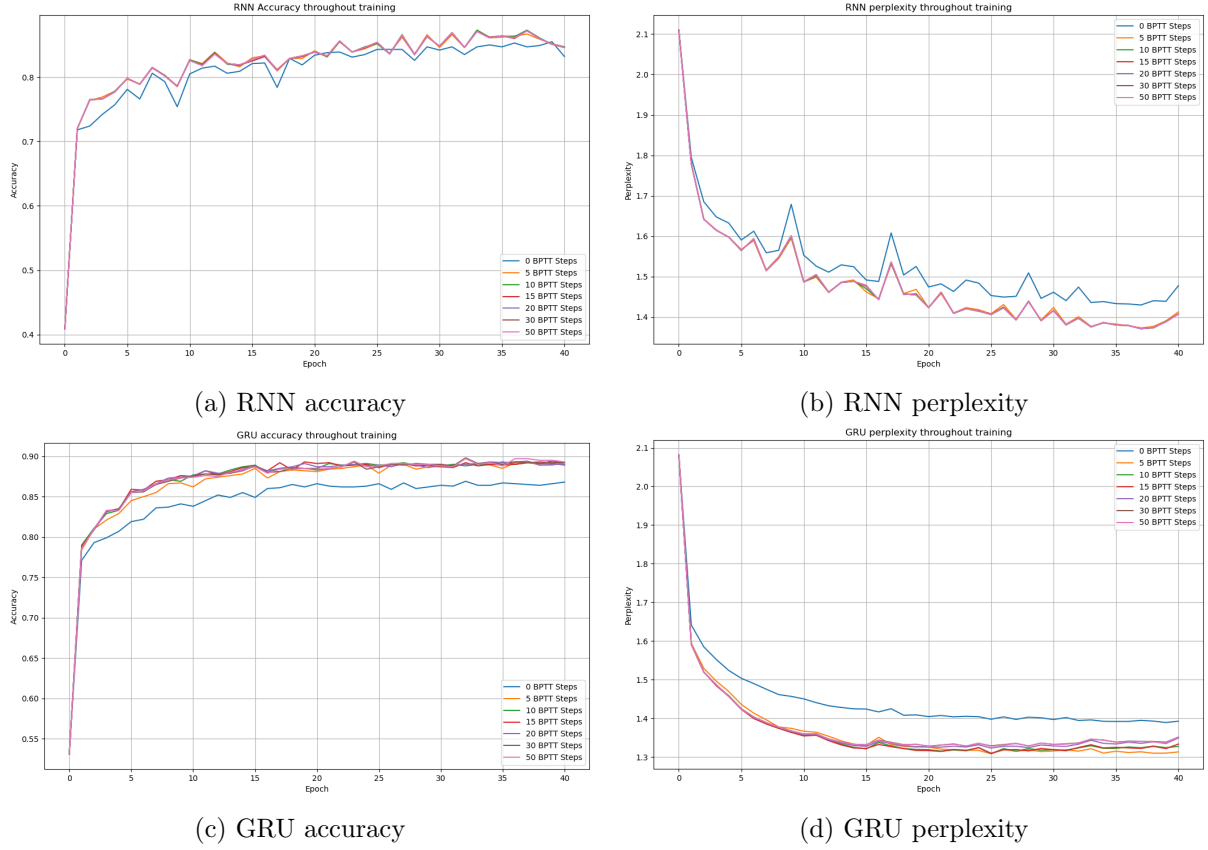


Figure 1: Training of the RNN and GRU with different BPTT steps.

Analysis of the accuracy and perplexity curves shows that GRU models consistently outperform their RNN counterparts across all metrics. Notably, the GRU curves demonstrate greater stability and reach optimal performance thresholds significantly earlier in the training process than RNNs. Despite varying BPTT step sizes, no RNN configuration achieved performance levels comparable to any GRU setting, emphasizing the substantial advantage of GRUs in handling vanishing gradients. This superior performance and faster convergence are likely attributable to GRUs' enhanced ability to mitigate the effects of vanishing gradients, a common challenge in sequence learning with deep networks. Figure 2 illustrates this phenomenon, showing the percentage change in parameters over epochs for the 50-step BPTT configuration, further highlighting the GRUs' efficiency in parameter updates.

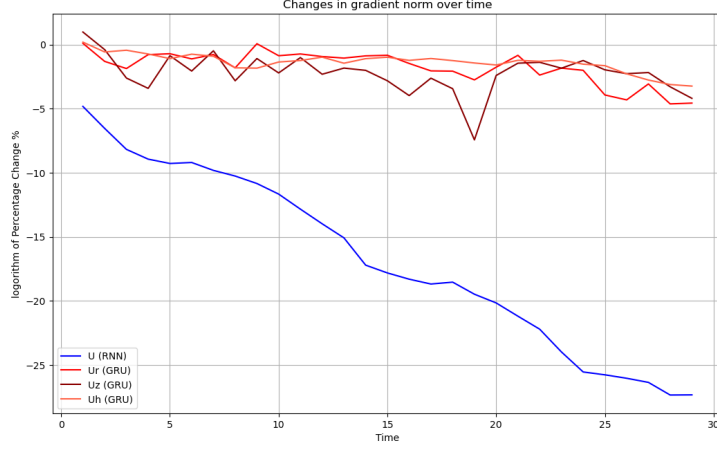


Figure 2: The percentage changes in the gradient norm over time.

3.3.2 Experiment 2

Performance Analysis

The Baseline GRU model exhibited the lowest perplexity and the highest accuracy among all models, demonstrating its effectiveness in capturing long-term dependencies and managing state transitions efficiently. This outcome is expected, as GRUs selectively retain relevant past information while discarding unnecessary details through the update and reset gates.

In contrast, the standard RNN consistently showed higher perplexity and lower accuracy, highlighting its struggle with learning sequence dependencies due to the absence of explicit gating mechanisms.

Disabling either the update gate (`update_off`) or the reset gate (`reset_off`) resulted in higher perplexity and lower accuracy compared to the Baseline GRU. However, these models still outperformed both the standard RNN and the configuration where both gates were disabled (`both_off`).

Among the two, the `reset_off` configuration performed better than `update_off`, indicating that the update gate plays a more critical role in model performance. This distinction arises because:

- The reset gate controls how much past information is forgotten.
- The update gate controls how much new information is incorporated.

When the update gate is removed, the model struggles to regulate information retention, leading to uncontrolled state propagation. Its absence causes the model to overly rely on short-term patterns, resulting in inconsistent predictions and increased perplexity.

In contrast, without the reset gate, although the model has difficulty efficiently forgetting irrelevant information, it can still integrate new information in a structured manner. This makes the `reset_off` configuration superior to `update_off` in overall performance.

The GRU without both gates (`both_off`) performs the worst among all models, even worse than a standard RNN. While `both_off` functions similarly to an RNN in that both lack a method to regulate memory updates and struggle with long-term dependency learning, its extra parameters

make training more complex, leading to lower performance. Unlike an RNN, which has only one weight matrix (U), a GRU retains additional weight matrices (U_r, U_z, U_h). This increased parameter count requires more optimization, resulting in a less efficient training process.

The performance analysis results are illustrated in Figure 3, which presents accuracy and perplexity across different models.

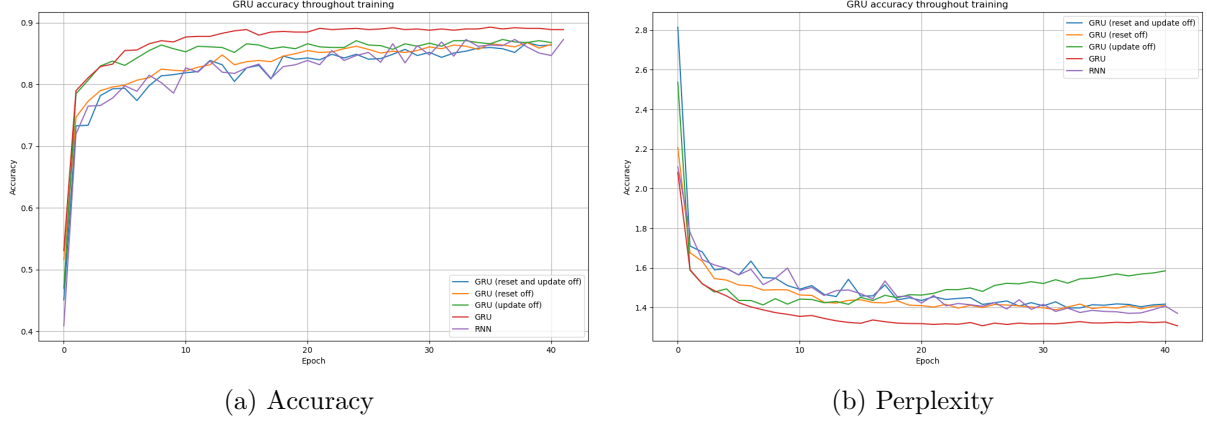


Figure 3: Performance Evaluation of RNN, GRU, and Gate Variants of GRU

Gradient-Based Analysis

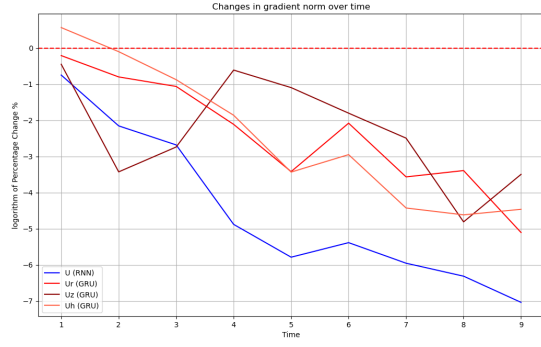
The gradient change analysis offers further insight into the observed performance differences by showing how each model updates its weights over time.

- The standard RNN exhibits vanishing gradients, making it difficult to retain long-term dependencies.
- The GRU maintains stable gradients, ensuring controlled memory updates.
- The `update_off` and `reset_off` configurations show similar trends, but `reset_off` is slightly more stable, meaning the update gate is more critical for maintaining controlled learning.
- The GRU without both gates (`both_off`) experiences uncontrolled gradient growth, proving that both gates are crucial for maintaining balanced learning.

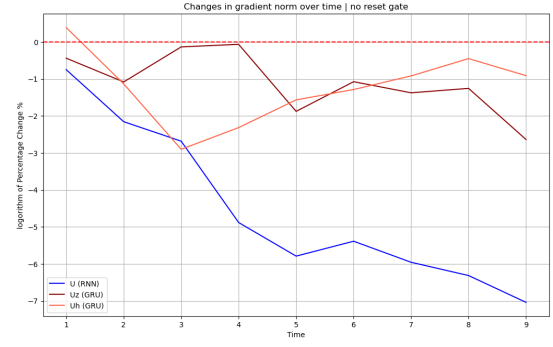
Since `both_off` retains GRU’s additional parameters but lacks mechanisms to regulate them, training becomes unstable, leading to chaotic learning. Unlike an RNN, which at least follows a structured update process, `both_off` propagates information indiscriminately, preventing convergence. The gradient analysis confirms that GRU’s power comes from its gates, not just its structure, as removing them leads to higher instability rather than improved learning.

Moreover, when the reset gate is disabled (`reset_off`), the U_r values remain unchanged, indicating that the reset mechanism is inactive. Similarly, when the update gate is disabled (`update_off`), the U_z values do not update, confirming that the model is not incorporating new information through the update mechanism. This demonstrates that while GRUs retain additional weight matrices, these parameters provide no benefit when their corresponding gates are disabled, leading to unnecessary computational overhead without improving performance.

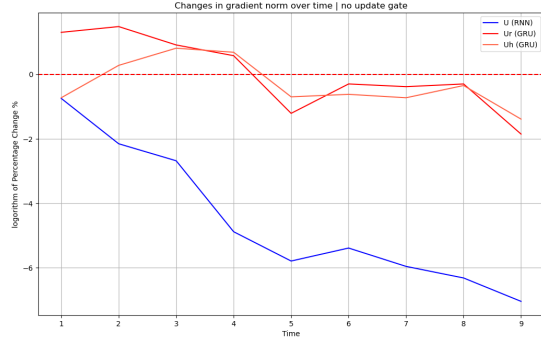
The gradient analysis results are illustrated in Figure 4, which visualizes the gradient evolution across different models.



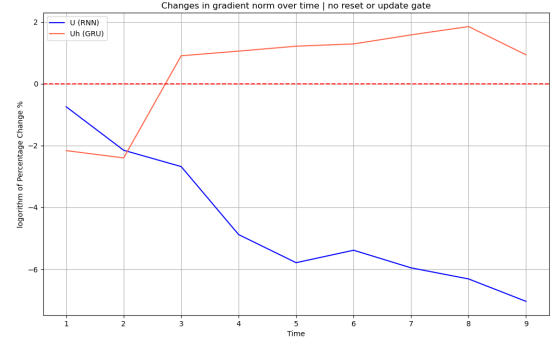
(a) Gradient flow - Baseline GRU



(b) Gradient flow - No Reset Gate



(c) Gradient flow - No Update Gate



(d) Gradient flow - No Reset or Update Gate

Figure 4: Gradient Analysis of RNN, GRU, and Variants with Disabled Gates

References

- [1] Pedro Freire, Sasipim Srivallapanondh, Antonio Napoli, Jaroslaw Prilepsky, and Sergei Turtitsyn. Computational complexity evaluation of neural network applications in signal processing. June 2022.

A Results Table for Section 2

	Hidden	Initial	Ep. 1	Ep. 2	Ep. 3	Ep. 4	Ep. 5	Ep. 6	Ep. 7	Ep. 8	Ep. 9	Ep.10
RNN	10	0.6486	0.6385	0.6351	0.6315	0.6290	0.6291	0.6238	0.6218	0.6225	0.6181	0.6161
	25	0.6586	0.6445	0.6416	0.6465	0.6277	0.6250	0.6169	0.6135	0.6108	0.6100	0.6097
	50	0.7466	1.0743	0.7008	0.6374	0.6238	0.6341	0.6099	0.6252	0.6037	0.5991	0.5975
GRU	10	0.6957	0.6742	0.6590	0.6471	0.6361	0.6265	0.6182	0.6100	0.6033	0.5972	0.5918
	25	0.8096	0.6600	0.6153	0.5919	0.5781	0.5692	0.5626	0.5577	0.5533	0.5499	0.5467
	50	0.7328	0.6041	0.5769	0.5615	0.5511	0.5433	0.5379	0.5334	0.5303	0.5266	0.5234

Table 11: Validation Loss of RNN and GRU with BPTT Step = 0

	Hidden	Initial	Ep. 1	Ep. 2	Ep. 3	Ep. 4	Ep. 5	Ep. 6	Ep. 7	Ep. 8	Ep. 9	Ep.10
RNN	10	0.659	0.659	0.659	0.659	0.659	0.659	0.659	0.659	0.659	0.659	0.659
	25	0.656	0.658	0.659	0.659	0.662	0.664	0.666	0.667	0.667	0.667	0.688
	50	0.409	0.659	0.668	0.668	0.669	0.669	0.671	0.669	0.692	0.692	0.687
GRU	10	0.511	0.601	0.628	0.655	0.659	0.666	0.663	0.669	0.667	0.668	0.672
	25	0.380	0.641	0.693	0.700	0.713	0.718	0.721	0.725	0.730	0.732	0.734
	50	0.531	0.674	0.703	0.710	0.725	0.738	0.742	0.747	0.746	0.747	0.751

Table 12: Prediction Accuracy of RNN and GRU with BPTT Step = 0

	Hidden	Ep. 1	Ep. 2	Ep. 3	Ep. 4	Ep. 5	Ep. 6	Ep. 7	Ep. 8	Ep. 9	Ep.10	Total
RNN	10	1.41	1.37	1.46	1.45	1.43	1.41	1.41	1.41	1.46	1.78	15.20
	25	2.12	2.12	2.25	2.56	2.45	2.46	2.46	2.50	2.49	2.56	24.92
	50	4.09	3.71	3.08	3.27	5.79	4.32	5.45	5.44	3.40	5.51	45.33
GRU	10	3.34	5.70	4.24	3.62	3.83	3.83	3.60	3.68	3.88	3.65	40.85
	25	6.71	7.23	7.69	7.65	7.29	6.71	6.70	6.67	6.67	6.77	72.93
	50	7.57	8.29	8.23	8.22	8.16	8.94	8.33	8.23	8.28	8.47	85.78

Table 13: Training Time of RNN and GRU with BPTT Step = 0

B Computational Complexity of RNN and GRU with Respect to Hidden Units Size

1. Recurrent Neural Networks (RNN)

For an input sequence of length T , input dimension d , the hidden state update in a simple RNN is given by:

$$s_t = \sigma(Vx_t + Us_{t-1})$$

The computational complexity per time step comes from matrix multiplications [1]:

$$\begin{aligned} Us_{t-1} &: O(s^2) \\ Vx_t &: O(sd) \end{aligned}$$

For a full sequence of length T , the total forward pass computation is:

$$O(T(s^2 + sd))$$

2. Gated Recurrent Unit (GRU)

A GRU includes additional gating mechanisms:

$$z_t = \sigma(V_z x_t + U_z s_{t-1})$$

$$r_t = \sigma(V_r x_t + U_r s_{t-1})$$

$$\tilde{h}_t = \tanh(V_h x_t + U_h(r_t \odot s_{t-1}))$$

$$s_t = (1 - z_t) \odot s_{t-1} + z_t \odot \tilde{h}_t$$

Each gate and candidate hidden state involves additional matrix multiplications [1]:

$$\begin{aligned} V_z x_t, U_z s_{t-1} &: O(sd + s^2) \\ V_r x_t, U_r s_{t-1} &: O(sd + s^2) \\ V_h x_t, U_h(r_t \odot s_{t-1}) &: O(sd + s^2) \end{aligned}$$

Since GRU performs three such computations per time step, the total computational complexity per sequence is:

$$O(T(3s^2 + 3sd))$$

3. Comparison and Effect on Training Time

Since both models scale with $O(s^2)$, increasing s significantly increases training time, especially for longer sequences T . The GRU has three times the computational cost of a simple RNN due to its additional gates. Therefore, for large s , GRU takes proportionally longer to train compared to RNN.

Thus, the overall complexity equations are:

$$\text{RNN: } O(T(s^2 + sd))$$

$$\text{GRU: } O(T(3s^2 + 3sd))$$

These equations demonstrate how increasing the number of hidden units increases computational cost and training time.